

Gradient Descent

בשביל למצוא את המינימום של $f(x)$ נרצה ללכת במורד הגרדיאנט של פונקציית הloss. עושים את זה לפי:

$$w^t = w^{t-1} - \eta \nabla L(w^{t-1})$$

הפרמטר η נקרא "learning rate" או "step size". אנחנו רוצים לבחור learning rate שלא יהיה קטן מדי - כי אז יקח לנו יותר מדי זמן (ויותר מדי דוגמאות) להתכנס, וגם כי אז אנחנו עלולים להגיע לlocal minimum. מצד שני, אם ניקח learning rate גדול מדי אנחנו עלולים "לדלג" מעל המינימום שאנחנו מחפשים.

הגדרות: • פונקציה היא β -smooth (או Lipschitz continuous) אם לכל $x, y \in \mathbb{R}^d$:

$$\|f(x) - f(y)\| \leq \beta \|x - y\|$$

כלומר - הפונקציה לא זזה יותר מדי.

• פונקציה f היא קמורה ("convex") אם

$$f(y) \geq f(x) + \nabla f(x)^T (y - x)$$

משפט: תהי $f: \mathbb{R}^d \rightarrow \mathbb{R}$ פונקציה וגזירה, ושהגרדיאנט שלה הוא Lipschitz continuous עם קבוע $\beta > 0$. אזי אם נריץ gradient descent במשך T צעדים עם גודל צעד $\eta \leq \frac{1}{\beta}$ נקבל פתרון x^T המקיים:

$$f(x^T) - f(x^*) \leq \frac{\|x^0 - x^*\|^2}{2\eta T} \leq \epsilon$$

כאשר $f(x^*)$ הוא הערך האופטימלי.

אינטואיטיבית: זה אומר שירידה בגרדיאנט תגרום לפונקציה להתכנס בקצב $O(\frac{1}{T})$.

למה: לכל פונקציה β -חלקה, אפשר לחסום אותה באמצעות חסם עליון ריבועי:

$$f(y) \leq f(x) + \nabla f(x)^T (y - x) + \frac{\beta}{2} \|y - x\|^2$$

אפשר לצבור את הגרדיאנט של כל training setn בשביל לעדכן פרמטרים:

$$w^t = w^{t-1} - \eta \frac{1}{n} \sum_{i=1}^n \nabla_w \ell(x_{i_t}, y_i, w^{t-1})$$

אבל זה איטי. לכן נרצה:

Stochastic Gradient Descent

לכל איטרציה t , בוחרים דוגמא אקראית (x_{i_t}, y_{i_t}) ומעדכנים:

$$w^t = w^{t-1} - \eta \nabla_w \ell(x_{i_t}, y_{i_t}, w^{t-1})$$

יש לנו ככה הרבה יותר צעדים - אבל כל צעד הוא מאוד זול (כי משתמשים רק בדוגמה אחת)
הבעיה בשיטה הזאת שהיא פחות מדויקת, ושגם יותר קשה למקבל אותה עם GPU.

Mini-Batch Gradient Descent

אפשר לבחור משהו באמצע - לא את כל הנקודות אבל יותר מנקודה אחת - נבחר בכל צעד k דוגמאות $\mathcal{B}_t \subset \{1, \dots, n\}$ ונעדכן לפיהן:

$$w^t = w^{t-1} - \eta \frac{1}{k} \sum_{i \in \mathcal{B}_t} \nabla_w \ell(x_i, y_i, w^{t-1})$$

$|\mathcal{B}_t| = k$ נקרא "batch size" - בד"כ יהיה 32, 64 או 128.

רגרסיה לינארית

לעשות ממוצע נאיבי זה בד"כ לא טוב - גם בהנחה שכל הפיצרים מנורמלים עדיין יש פיצ'רים יותר ופחות חשובים, ורוצים לעשות משקלים.

תזכורת: יש לנו נתונים - דוגמאות מתוייגות $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^n$ כאשר $x_i \in \mathbb{R}^d, y_i \in \mathbb{R}$ רוצים למצוא פונקציה $f: \mathbb{R}^d \rightarrow \mathbb{R}$ שממפה כל דוגמה x_i ל $\hat{y}_i = f(x_i)$.

ברגרסיה לינארית נרצה ש f תהיה פונקציה לינארית $f(x_i) = x_i^T \beta$ כאשר $\beta \in \mathbb{R}^d$ הוא ווקטור של מקדמים. או בצורת מטריצה: $\hat{y} = X\beta$ כאשר $X \in \mathbb{R}^{n \times d}$ מייצג את $\mathcal{X}$.

עושים זאת באמצעות מזעור ה Mean Squared Error:

$$MSE := \frac{1}{n} (y - X\beta)^T (y - X\beta)$$

הפתרון שממזער את ה MSE הוא

$$\bar{\beta} = (X^T X)^{-1} X^T y$$

רגרסיה לינארית בתור ERM

יש לנו רגרסיה לינארית

$$\hat{y}_w(x) = w \cdot x$$

נרצה למזער את ה loss:

$$\ell(X, Y, w) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (y_i - \hat{y}_w(x_i))^2$$

אם נגזור לפי w נקבל:

$$\frac{\partial \ell}{\partial w} = \dots = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_w(x_i)) (-x_i)$$

אפשר להפוך את זה ל gradient descent:

$$w^t \leftarrow w^{t-1} - \eta \cdot \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_w(x_i)) (-x_i)$$

האלגוריתם: 1. בוחרים נקודת התחלה אקראית - פרמטרים w^0 .
2. חוזרים:

- מנבאים תגיות $\hat{y}_w(x)$
- מחשבים את הגרדיאנט של ה loss.
- מעדכנים את הפרמטרים באמצעות gradient descent.

Linear Classifiers

Linear Binary Classifiers and Perceptron

מרחב ההיפותזות שלנו הוא:

$$h_{w,b}(x) = \begin{cases} +1 & w^T x > b \\ -1 & w^T x < b \end{cases}$$

זה בעצם מסווג לנו מה מעל הקו ומה מתחת לקו.

SGD עבור Single Neuron Model

אפשר לעדכן את הפרמטרים של הפרספטרון כאשר בכל צעד, אם $y_i w^T x_i < 0$, מעדכנים $w \leftarrow w + \eta y_i x_i$.

הערה: b כאן הוא חלק מ- w - כלומר "מגדילים" את w ומסמנים $b = 0$.

איך עושים לזה רגולריזציה? מוסיפים איבר רגולריזציה $\frac{\lambda}{2} \|w\|^2 = \frac{\lambda}{2} w^T w$, כלומר:

$$\ell(w; x, y) := \ell_{\text{hinge}}(w; x, y) + \frac{\lambda}{2} \|w\|^2 = \begin{cases} -y w^T x + \frac{\lambda}{2} w^T w & y w^T x < 0 \\ \frac{\lambda}{2} w^T w & y w^T x > 0 \end{cases}$$

ואז בכל עדכון, אם $y_i w^T x_i < 0$ נעדכן $w \leftarrow w - \eta (\lambda w - y_i x_i)$ ואם $y_i w^T x_i > 0$ נעדכן $w \leftarrow w - \eta (\lambda w)$ (כי הסיווג נכון, אבל רוצים עדיין לעדכן לפי הרגולריזציה)